

Incident Report

FastAPI Service – HTTP 5xx Error Incident

1. Incident 개요 (Executive Summary)

본 문서는 FastAPI 기반 HTTP API 서비스에서 발생한 **HTTP 5xx 오류 증가 상황**을 대상으로 수행한 **의도적 장애 재현 실험**에 대한 Incident Report이다.

본 Incident는 실제 운영 환경에서 발생할 수 있는 서버 측 오류 상황을 모사하여, **사전에 정의한 SLI/SLO 기반 Alerting, Incident Response, Recovery, RCA 흐름이 의도한 대로 동작하는지를 검증하는 것**을 목적으로 한다.

본 실험에서 장애는 통제된 방식으로 주입되었으며, 데이터 손실이나 인프라 장애 없이 안전하게 복구되었다.

2. Incident 기본 정보

항목	내용
Incident 유형	Reliability-impacting incident
서비스	fastapi-app
장애 유형	Server-side failure
영향 지표	Availability SLI, Error Rate SLI
감지 방식	Prometheus Alert Rule
대응 방식	수동 대응 (Runbook 기반)
장애 성격	Controlled Failure Injection

3. Incident 목적 (Purpose)

본 Incident의 목적은 다음과 같다.

- 실제 운영 환경에서 발생 가능한 단일·현실적인 장애 시나리오 재현
- 사전에 정의한 SLI/SLO 기반 Alert가 정상적으로 트리거되는지 검증
- 장애 발생 → 감지 → 대응 → 복구까지의 Incident Response 흐름 검증
- Alert가 단순 임계치 초과가 아닌 SLO 보호 신호로 동작하는지 확인

본 문서는 장애 자체보다, 장애를 어떻게 인지하고 관리했는지에 초점을 둔다.

4. 장애 시나리오 (Incident Scenario)

4.1 선택한 장애 시나리오

- **시나리오:** API 서버에서 HTTP 5xx 오류가 지속적으로 발생하는 상황
- **장애 유형:** Server-side failure
- **의도**
 - Availability SLI 영향 확인
 - Error Rate SLI 기반 Alert 검증

본 시나리오는 사용자 관점에서 “요청이 실패한다”는 증상이 명확하며, Availability SLO 및 Error Rate SLI와 직접적으로 연결되는 대표적인 장애 유형이다.

본 실험에서는 시스템 복잡도를 최소화하기 위해 단일 장애 시나리오만을 사용하였다.

5. 장애 유도 (Failure Injection)

5.1 장애 유도 방식

장애는 FastAPI 서비스의 /error 엔드포인트를 활용하여 항상 HTTP 500을 반환하도록 설계된 요청을 일정 시간 반복 호출하는 방식으로 유도되었다.

- 장애 원인이 명확하고 통제 가능함
- 실험 종료 즉시 정상 상태로 복구 가능
- 동일한 docker-compose 환경에서 재현 가능

이는 실제 운영 장애를 모사하되, 시스템에 영구적인 영향을 주지 않는 안전한 장애 주입 방식이다.

5.2 기대 효과

- HTTP 5xx 증가 → Error Rate SLI 상승
 - Success Rate 감소 → Availability SLO 영향
 - Error Rate 기반 Prometheus Alert Rule 트리거
-

6. 장애 감지 (Detection)

6.1 감지 수단

본 Incident는 **Prometheus Alert Rule(HighErrorRate)**을 통해 감지되었다.

Alert는 HTTP 5xx 비율이 5%를 초과한 상태가 2분 이상 지속될 경우 발생하도록 설계되었다.

이는 일시적인 오류로 인한 **Alert noise**를 방지하고, 지속적인 신뢰성 위반만 감지하기 위한 설계이다.

6.2 감지 과정

- API 서버에서 HTTP 5xx 응답이 지속적으로 발생
- Error Rate SLI가 임계치를 초과
- Alert 상태 전이:

Inactive → Pending → Firing

6.3 감지 시간

장애 발생 시점 기준 약 N분 후 Alert가 Firing 상태로 전이되었다.

이 지연은 Alert Rule에 정의된 for 조건에 따른 의도된 동작이며, **Availability SLO 보호를 위한 정상적인 Alert 동작**으로 판단된다.

7. 영향 분석 (Impact Analysis)

7.1 사용자 영향

- 일부 API 요청이 HTTP 500 응답으로 실패
- 정상 응답을 기대하는 요청이 처리되지 않음
- 사용자 또는 호출 주체가 요청 실패를 즉시 인지 가능한 상태

본 장애는 클라이언트 오류가 아닌 **서버 측 처리 실패**로 분류된다.

7.2 SLO 영향

- **Availability SLO**
 - Success Rate 감소로 SLO 위반 또는 위반에 근접한 상태 발생
- **Error Budget**
 - Error Rate 증가로 Rolling 30일 기준 Error Budget 일부 소모

본 장애는 단일 이벤트 자체보다, **SLO 평가 기간 내 신뢰성 지표가 어떻게 악화되는지를 보여주는 사례**이다.

8. 대응 (Response)

8.1 초기 대응

Alert 발생 후 즉각적인 조치에 앞서, 서비스 상태 및 장애 범위를 우선 확인하였다.

- Grafana Dashboard 확인
- 단일 엔드포인트 장애 여부 확인
- 오류가 일시적 스파이크인지 지속 장애인지 판단

8.2 조치 내용

- 장애 유도 로직 확인
- 의도적 장애 주입 중단
 - /error 엔드포인트 호출 중지
 - 또는 API 서비스 재시작

조치는 **필요 최소한의 범위로 제한**하였다.

8.3 대응 원칙

- 수동 대응 (Runbook 기반)
- Blame-free 원칙 유지
- 문제 해결 및 학습 중심 대응

9. 복구 (Recovery)

9.1 복구 시점

장애 유도 중단 이후 API 서비스의 정상 응답이 확인되었고, Alert 상태는 **Firing** → **Resolved**로 전이되었다.

9.2 복구 판단 기준

복구 여부는 개별 요청이 아닌, **SLI/SLO 기준**으로 판단하였다.

- Error Rate SLI 정상 범위 복귀
- Availability 지표 안정화

서비스는 정의된 SLO 기준 내의 정상 상태로 복구되었다.

10. Root Cause Analysis (RCA)

10.1 직접 원인

- /error 엔드포인트가 항상 HTTP 500을 반환하도록 설계됨
- 반복 호출로 서버 오류 트래픽이 유지됨

10.2 근본 원인

본 Incident는 코드 결함이나 인프라 장애가 아니라, 신뢰성 검증을 위한 **통제된 장애 주입(Controlled Failure Injection)**의 결과이다.

11. Lessons Learned

11.1 잘 된 점

- **Error Rate 기반 Alert**가 Availability SLO 위반 징후를 조기에 감지
- Incident Response 전체 흐름 검증 완료
- **SLI/SLO 기반으로 일관된 판단** 가능

11.2 개선할 점

- 단일 지표 기반 Alert는 상황에 따라 **noise** 발생 가능
- SLI 중심 Alert는 **장기적 SLO 판단에 한계** 존재

12. 재발 방지 및 향후 계획

- SLO 기반 Alert 도입 (Error Budget burn rate)
- Multi-window / Multi-burn-rate Alert 적용
- **Latency 기반 장애 시나리오** 추가
- Incident 대응 절차 **Runbook**화

본 Incident Report는 단일 장애 사례를 통해, **SLI/SLO 기반 운영 판단과 신뢰성 중심 Incident 관리 흐름**이 실제로 어떻게 적용되는지를 검증한 결과를 정리한 문서이다.